From Chaos to Calm: Revamping Mobile Releases at Turo

A behind-the-scenes look at how we turned a fragile, all-day release process into a calm, automated process—one button closer to the dream

---

How Turo does mobile releases

Turo is the world's largest peer-to-peer car-sharing marketplace. Our mobile apps on iOS and Android are at the heart of how millions of guests find their perfect vehicle and how hosts share their cars. We're proud that our users consistently rate our apps highly, with ratings around **4.8 stars on the Apple App Store** and **4.9 stars on the Google Play Store**.

To keep delivering that great experience and quickly bring new features and improvements to our community, we release updates weekly. That weekly cadence is a big commitment, since each release is a version we can't easily retract or modify once it's sent. This post is a look at how we transformed our mobile release process from a manual marathon into a streamlined operation, making life better for our engineers and ensuring we can keep innovating for our users.

Friday used to mean chaos

Fridays were release days for the mobile engineering team, and they were often challenging. The process felt like a weekly firefighting drill, something that everyone quietly knew would consume their day, not because releases were complex, but because processes were manual.

Each week, a different mobile engineer took on the role of "Release Manager." This meant blocking off the entire day, preparing for constant context-switching, and manually navigating a long checklist.

Across both iOS and Android, the hurdles were similar:

- The `develop` branch had to be frozen, halting other engineering work.

- Engineers spent hours on manual QA checks, testing the same app workflows every single week.
- Tasks were coordinated by hand across a bunch of tools—GitHub, Jira, Confluence, and our CI/CD platforms. There was no single source of truth, and release manager rotations were tracked in a spreadsheet. Everyone had their own slightly different way of doing things, which made onboarding new release managers particularly painful.
- Hotfixes only added to the complexity, often leading to merge conflicts with the `develop` branch.

On top of these shared issues, each platform had its own unique complexities. For iOS, this included managing builds through Xcode Cloud and dealing with a complex even/odd versioning system, a clever workaround to ensure our internal test builds were always numbered ahead of the live app, but one that required careful manual tracking.

Meanwhile, on Android, the process involved manually kicking off multiple GitHub Actions, using Firebase App Distribution for test builds, and manually handling app bundle uploads and rollouts on the Google Play Console.

Fridays became less about confidently shipping new updates and more about navigating a tricky process.

Why we had to change

Weekly releases resulted in about 12 lost engineering hours, 6 each for iOS and Android. That's more than a full workday lost per week to repetitive manual tasks, time that could have gone into building new features our users would love.

Our mobile engineering team had grown quickly, but our release process hadn't evolved with it.

> As one of our mobile engineers quite aptly put it, *"We scaled to 15–20 engineers, and suddenly those informal rules just didn't work anymore."*

Each platform had its own slightly different way of doing things, and a lot of crucial knowledge lived only in individual engineers' minds.

So what were the recurring issues? When we dug in, the pattern was clear — too much manual work. The main culprits were:

- No clear, unified structure for releases. Documentation was scattered and often confusing, especially for newer engineers.
- App versioning between Android and iOS wasn't aligned, making it hard to track feature releases.
- Information was scattered across Jira, GitHub, and Confluence, with no single source of truth.
- The entire process was dominated by manual tasks, from freezing branches to submitting builds.

The burden of this outdated process was growing. Releases were becoming a bottleneck, and the toll on productivity, our consistency, and even team morale wasn't something we could ignore any longer. We needed automation, not more documentation.

What we did: the path from problem to solution

We knew things had to change; our manual release management just wasn't cutting it anymore. We briefly weighed building our own tools versus finding an existing solution. Given our goal of "push a button, do a release" and wanting to avoid the overhead of maintaining more internal software, we decided to look for external tools that could help us build a better process.

Our search led us to a couple of key services. We found Runway, a platform that could help orchestrate our releases and automate many of the repetitive steps by integrating with GitHub, Jira, and our CI/CD systems. We also partnered with Applause to manage our external QA testing through their crowdtesting platform.

With these tools in hand, we then rolled up our sleeves and made some significant changes:

- **Branching strategy:** We moved from the old `master`/`develop` model to Trunk-Based Development (TBD) with dedicated release branches. No more freezing `develop` or back-merging hotfixes, now fixes land in `develop` first, then get cherry-picked to a hotfix branch. This keeps our main codebase stable and up-to-date.
- **Better QA without slowing down:** We integrated Applause's crowdtesting to catch issues sooner, but kept our weekly shipping cadence. We shifted release

day from Friday to Thursday to give Applause time for checks without slowing us down. We provided them with detailed test cases and app flows, and this shift significantly reduced the manual QA load on our engineering team

- **Runway automations:** We leveraged Runway's orchestration to handle critical parts of the process:
    - Automatic release branches and version bumps.
    - Automated build submissions to the App Store and Google Play.
    - Automated Slack messages and dedicated channels for every release or hotfix.
    - Smarter staged rollouts that accelerate automatically if stability holds.
    - Integrated stability monitoring with New Relic, pausing rollouts automatically if metrics dip.

These changes replaced manual, error-prone steps with predictable, reliable automation. No more complex scripts, no more accidental user error. The new setup provided a shared source of truth: anyone on the team could check the Runway dashboard or Slack channels and instantly know the status of a release.

The transformation was dramatic. Releases became predictable and efficient. For us engineers, it felt like we'd traded a clunky, manual setup for a sleek, automated one. Our revamped process really does have that 'new car smell' — that feeling of everything being fresh, clean, and running smoothly. It made a challenging part of our week genuinely better.

The new normal: calm releases, big wins

So, what's life like now? The difference was clear almost immediately: the new branching strategy, Applause for QA, and all the Runway automations.

> *The biggest win? Engineers got a significant amount of time back. Releases no longer hijack entire days. Most weeks, keeping an eye on the release process doesn't require more than a couple of hours of focused attention. This means more time for what we love doing: building features and tackling interesting engineering challenges.*

More importantly, nobody dreads release duty anymore. We actually folded release responsibilities into our existing PagerDuty rotation, and it fits right in. What used to be a

full-day, attention-draining task became just another manageable part of a standard on-call shift.

We also finally aligned both iOS and Android on a shared versioning format—`{year}.{week}.0`. This simple change made it so much easier for everyone to quickly see what version is live, what's currently rolling out, and where to find more details.

Communication is also streamlined. Key updates and rollout progress are automatically shared in Slack, with dedicated channels keeping a neat history for each release.

The process is no longer hidden away in someone's browser tabs or a single person's memory; it's visible, standardized, and consistently calm. Here's a high-level look at our new weekly rhythm:

<weekly app release cycle diagram>

We still ship our releases weekly, but now with throughput effectively doubled, and with far fewer headaches and blockers.

> *Our release day is Thursday, and the "Release Pilot" (the on-call engineer) typically just kicks things off and lets the automations handle the heavy lifting, from branch cuts to build submissions and QA coordination, right through to production rollout, which now happens smoothly starting Monday.*

Key takeaways & the road ahead

Looking back on this whole journey, our biggest takeaway was that investing in proper automation was far more valuable than just trying to optimize our manual processes. We didn't just plug in a new tool; we seized the opportunity to rethink how we ship mobile software from end to end.

The real lesson? Sometimes the bravest thing we can do is admit that humans shouldn't be doing repetitive tasks that computers can handle more efficiently. Runway provided a strong framework and powerful automation capabilities that helped us drive clarity, ownership, and consistency, ultimately becoming a key part of the lasting value in our revamped process.

And we’re not stopping here; we’re always looking for ways to automate more. We’ve recently enhanced integrations between our release platform and GitHub Actions using AWS Lambda for more tailored workflow automations. We’re also exploring tighter automation with our external QA process to catch potential issues even earlier.

These are steady, incremental steps toward that ultimate goal we talked about: “push a button, do a release.” We’re not 100% there yet, but we’re closer than we’ve ever been.

Mobile releases are fundamentally different from backend deploys. There are no quick rollbacks once your app is on users’ devices. Standardizing release processes and adopting the right tools truly pays dividends in the long run. Today, our mobile team is much happier with the release process. We have a standard, trustworthy system in place, and there are far fewer surprises.

> *As one of our mobile engineers put it after these changes, “It’s like we landed on the moon all over again.” And that feeling of achieving something significant, of finally escaping the manual grind and making our daily work better, is what this was all about.*
